



Smart Driver Safety System

Image-processing project

C05

Supervisors:

Dr. Saher Abdellatif

Eng. Renad Kalil

Students:

Kerolos Elromany Naeem

202302674

Ayaallah Mustafa Ibrahim

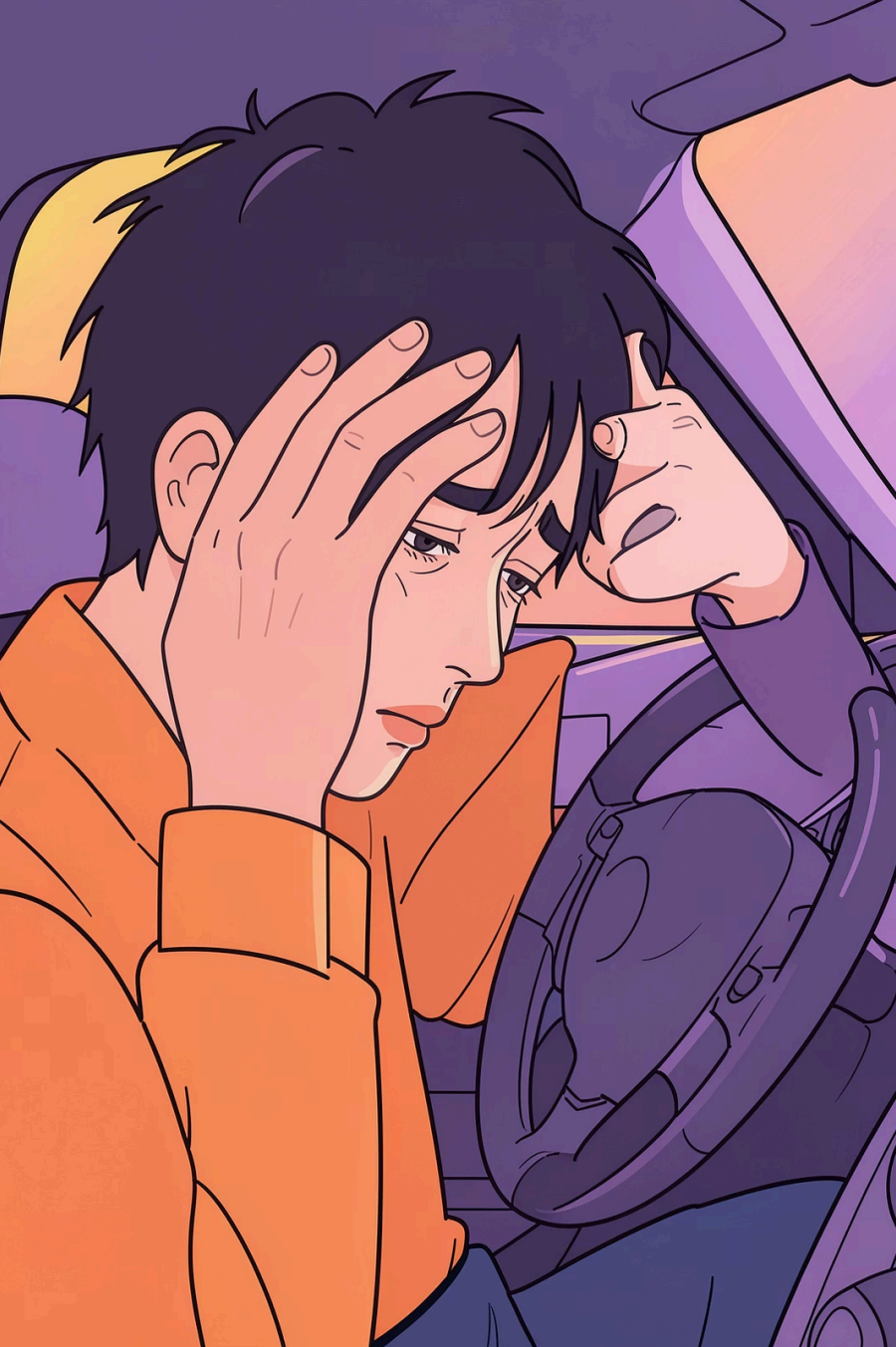
202301855

Amal Hatem Kamer

202302311

Salama Ahmed Salama

202302079



Problem Statement

PROBLEM STATEMENT

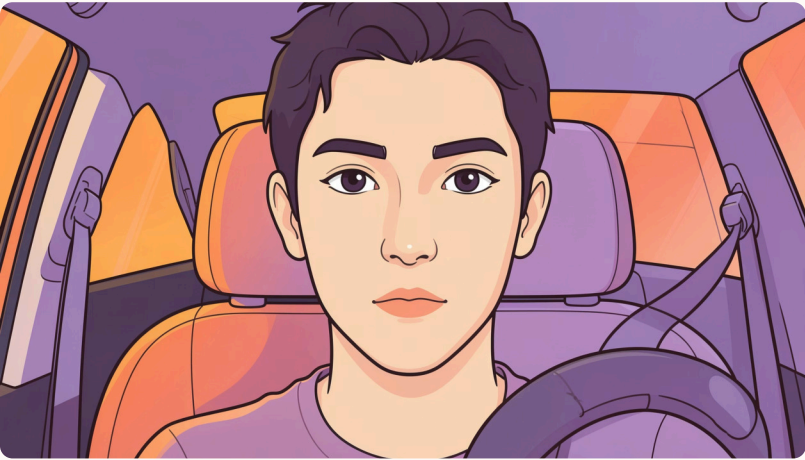
Driver drowsiness is one of the main causes of road accidents.

A reliable system is needed to automatically detect whether a driver is:

- Drowsy
- Non-Drowsy

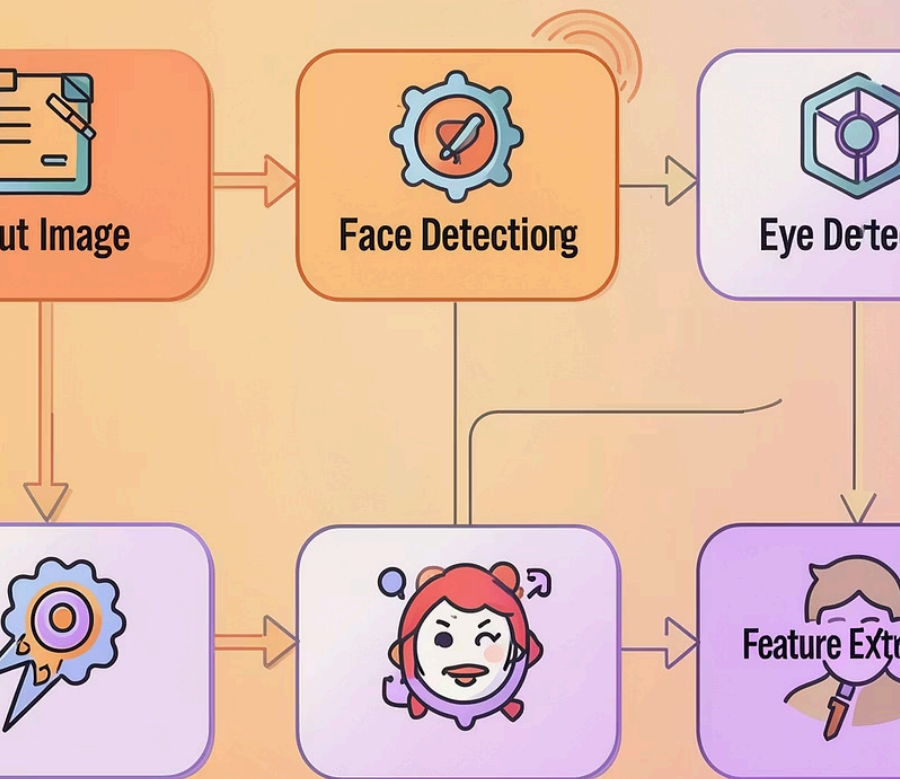


Goal: Reduce crash risk by enabling timely alerts or interventions.



Project Objectives

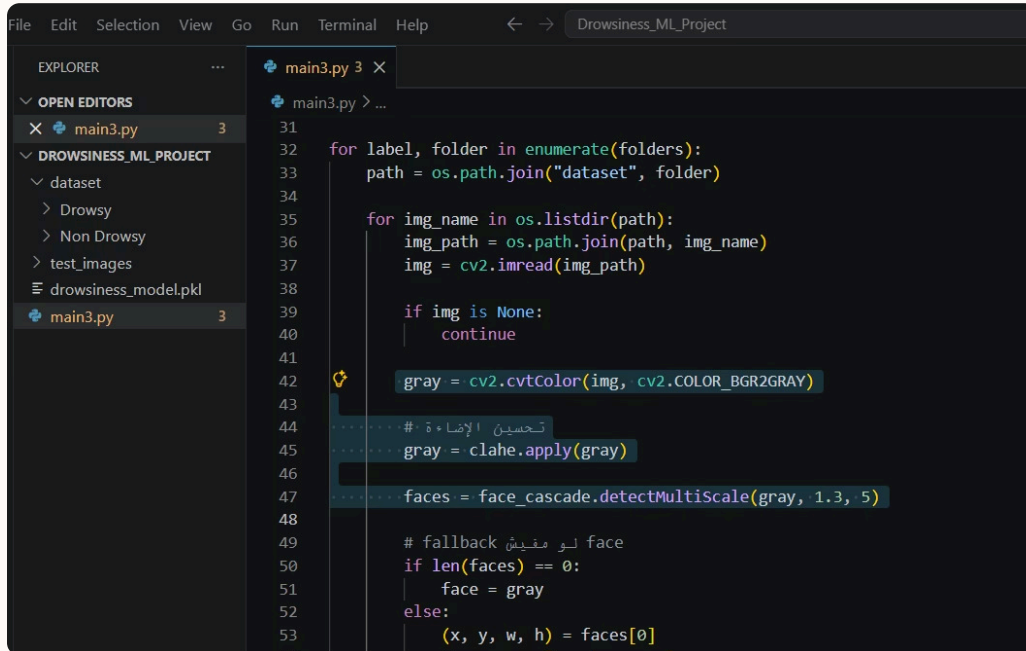
- The main objective of this project is to:
- Detect driver drowsiness using image processing
- Build a complete computer vision system
- Improve road safety



System Pipeline (High-level)

Input Image → Preprocessing → Face Detection → Eye Detection → Feature Extraction → Classification (SVM) → Output (Drowsy / Non-Drowsy)

Each stage is modular to allow component upgrades (e.g., replace SVM with CNN later).

A screenshot of a code editor window titled 'Drowsiness_ML_Project'. The left sidebar shows the 'EXPLORER' view with a file tree containing 'main3.py', 'dataset', 'Drowsy', 'Non Drowsy', 'test_images', and 'drowsiness_model.pkl'. The 'main3.py' file is open in the editor, showing Python code for image preprocessing. The code includes imports for 'os', 'cv2', and 'face_cascade'. It iterates through folders in a 'dataset' directory, reads images, converts them to grayscale, applies CLAHE for contrast enhancement, and uses a Haar cascade classifier to detect faces. A fallback mechanism is also shown for cases where no faces are detected.

```
31
32 for label, folder in enumerate(folders):
33     path = os.path.join("dataset", folder)
34
35     for img_name in os.listdir(path):
36         img_path = os.path.join(path, img_name)
37         img = cv2.imread(img_path)
38
39         if img is None:
40             continue
41
42         gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
43
44         # تحسين الإضاءة
45         gray = clahe.apply(gray)
46
47         faces = face_cascade.detectMultiScale(gray, 1.3, 5)
48
49         # fallback لو مفيش face
50         if len(faces) == 0:
51             face = gray
52         else:
53             (x, y, w, h) = faces[0]
```

Preprocessing

- Techniques applied:
- Grayscale conversion
- CLAHE (improves lighting conditions)
- Gaussian Blur (reduces noise)

Purpose:

- Enhance image quality
- Improve detection accuracy

```
44 # تحسين الإضاءة
45 gray = clahe.apply(gray)
46
47 faces = face_cascade.detectMultiScale(gray, 1.3, 5)
48
49 # fallback لو مفيش face
50 if len(faces) == 0:
51     face = gray
52 else:
53     (x, y, w, h) = faces[0]
54     face = gray[y:y+h, x:x+w]
55
56 face = cv2.GaussianBlur(face, (5, 5), 0)
57
58 eyes = eye_cascade.detectMultiScale(face)
59
60 # fallback لو مفيش عين
61 if len(eyes) == 0:
62     eye = cv2.resize(face, (50, 50))
63 else:
64     (ex, ey, ew, eh) = eyes[0]
65     eye = face[ey:ey+eh, ex:ex+ew]
66     eye = cv2.resize(eye, (50, 50))
67
68 # Edge Detection
69 edges = cv2.Canny(eye, 50, 150)
70
```

Detection Strategy

- **Face detection:** Haar cascade classifier — lightweight and real-time capable
- **Eye detection:** localized search within detected face region to reduce false positives
- **Fallbacks:**
 - If no face → use full image

If no eyes → use face region


```
main3.py 3
63     else:
64         (ex, ey, ew, eh) = eyes[0]
65         eye = face[ey:ey+eh, ex:ex+ew]
66         eye = cv2.resize(eye, (50, 50))
67
68     # Edge Detection
69     edges = cv2.Canny(eye, 50, 150)
70
71     # Data Augmentation
72     flipped = cv2.flip(edges, 1)
73
74     data.append(edges.flatten())
75     labels.append(label)
76
77     data.append(flipped.flatten())
78     labels.append(label)
79
80 data = np.array(data)
81 labels = np.array(labels)

```

PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS

OUTLINE

images (6).jpg -> Drowsy

Feature Extraction

- Resize image to 50×50
- Apply Edge Detection (Canny)
- Convert image to feature vector

Purpose:

- Convert image into numerical data feature vectors for fast SVM inference and smaller training sets.

Classification

Model Used:

- Support Vector Machine (LinearSVC)

Output:

- Drowsy
- Non-Drowsy.

```
3) Train or Load Model
=====
del_path = "drowsiness_model.pkl"

os.path.exists(model_path):
    model = joblib.load(model_path)
    print("✅ Model loaded")
else:
    print("🚀 Training started...")
    model = LinearSVC(max_iter=10000)
    model.fit(X_train, y_train)
    print("✅ Training finished!")

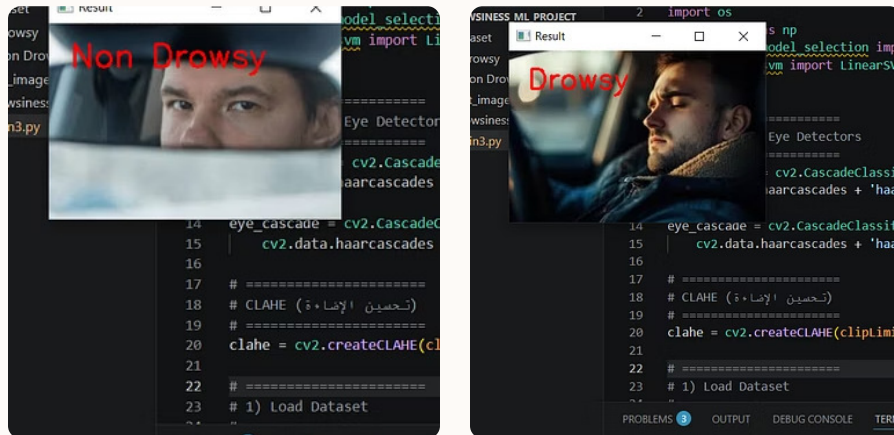
    joblib.dump(model, model_path)
    print("💾 Model saved")

=====
4) Evaluation

3  OUTPUT  DEBUG CONSOLE  TERMINAL

6).jpg -> Drowsy
7).jpg -> Drowsy
```


Results — Detection & Examples



The system:

- Detects face
- Detects eye
- Displays classification result

Examples:

- Drowsy
- Non-Drowsy

(Insert images here).

COMPARISON

A comparison was performed between:

Method	Accuracy
Without CLAHE	~0.46
With CLAHE	~0.50

Conclusion:

- CLAHE improves contrast
- Enhances system performance

CHALLENGES

The system faces some limitations:

- Lighting variations
- Face angle differences
- Eye detection failure in some cases

FUTURE WORK

Possible improvements:

- Use CNN (Deep Learning)
- Increase dataset size
- Real-time detection using camera

CONCLUSION

We successfully developed a complete system that:

- Uses multiple image processing techniques
- Applies machine learning for classification
- Provides real-time-like results